

SnapVM

Orquestração Autônoma e Gestão
de Estado para Agentes de IA



O que é o SNAPVVM?

O Problema

Agentes de IA em loops *ReAct* e pipelines automatizados frequentemente corrompem ambientes *stateful*.

Recuperação tradicional (Git/Docker) é lenta e ignora o estado da memória e processos vivos.

A Proposta

Uma ferramenta experimental de orquestração construída sobre o motor **AWS Firecracker**. Utilizamos snapshots de hardware para "rebobinar" o sistema instantaneamente para um estado saudável completo.

| Definição do MVP Experimental

- ✓ Cenário: Express API + PostgreSQL.
- ✓ Health Check: Gatilho automático para snapshots.
- ✓ Fallback: SnapVM como camada de fidelidade física.



Marcos da **Sprint 1**

Validação

Provamos o "Git Stateful Flaw": comandos tradicionais não restauram estados de memória ou bancos de dados PostgreSQL. Foco: Infraestrutura determinística e baseline de métricas.

Live LLM

Introdução de Agente Real no **Loop ReAct**. Resultados práticos em latência de restore (6.57s para 0.18s). Foco: Redução drástica de **Penalty Time** e **consumo de tokens** de API.

Orquestrador Potencializado por IA

O Cérebro da Operação

O **orquestrador SnapVM** atua como a camada de decisão crítica, mediando a interação entre a IA e a infraestrutura Firecracker.

Nesta fase, implementamos **validações básicas** que permitem ao orquestrador antecipar falhas de ambiente antes que o **agente perca contexto**.

Mecânicas de Orquestração



Health Checks

Validação contínua da saúde do ambiente via endpoint /health, garantindo que snapshots só ocorram em estados íntegros.



Snapshots LKG

Política de "Last Known Good": o sistema mantém automaticamente o último estado saudável para rollback garantido.



Rollback Ativo

Capacidade de rebobinar o sistema em milissegundos, eliminando o "hallucination retry loop" do agente de IA.

Gestão de **Escopo e Viés**

100%

Foco Experimental

Reconhecendo a Complexidade

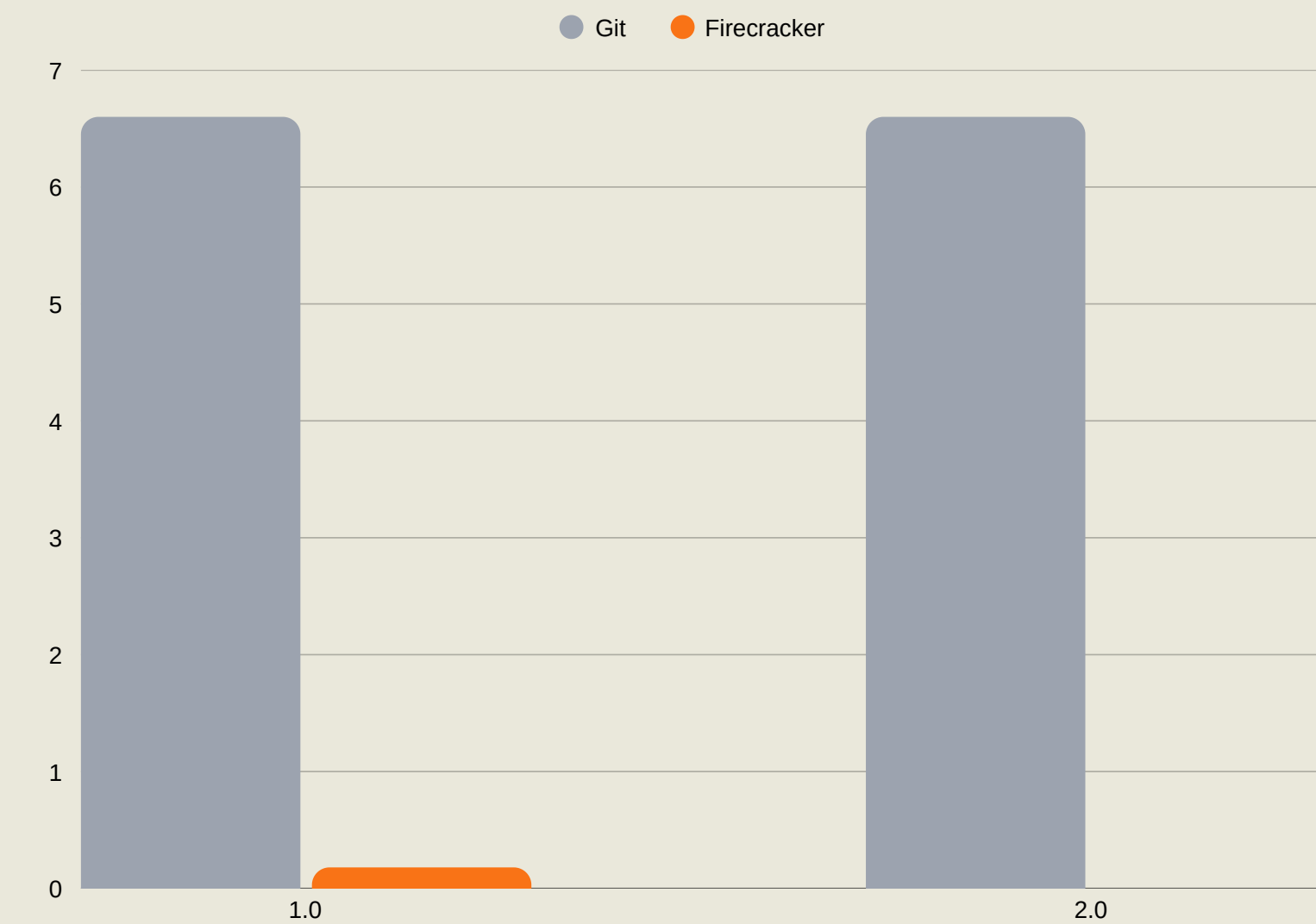
Assumimos que o SnapVM lida com uma arquitetura de alta escala e múltiplos vieses técnicos.

Decidimos seguir a estratégia de **MVP Experimental** citada anteriormente: um teste de projeto controlado que permite **aumentar o escopo gradualmente**, sem comprometer a integridade científica do estudo de caso.

Experimentos

Exp 1 & 2: Validação Base

V1 (mock determinístico, n=10) → V2 (LLM real ReAct loop, n=20)

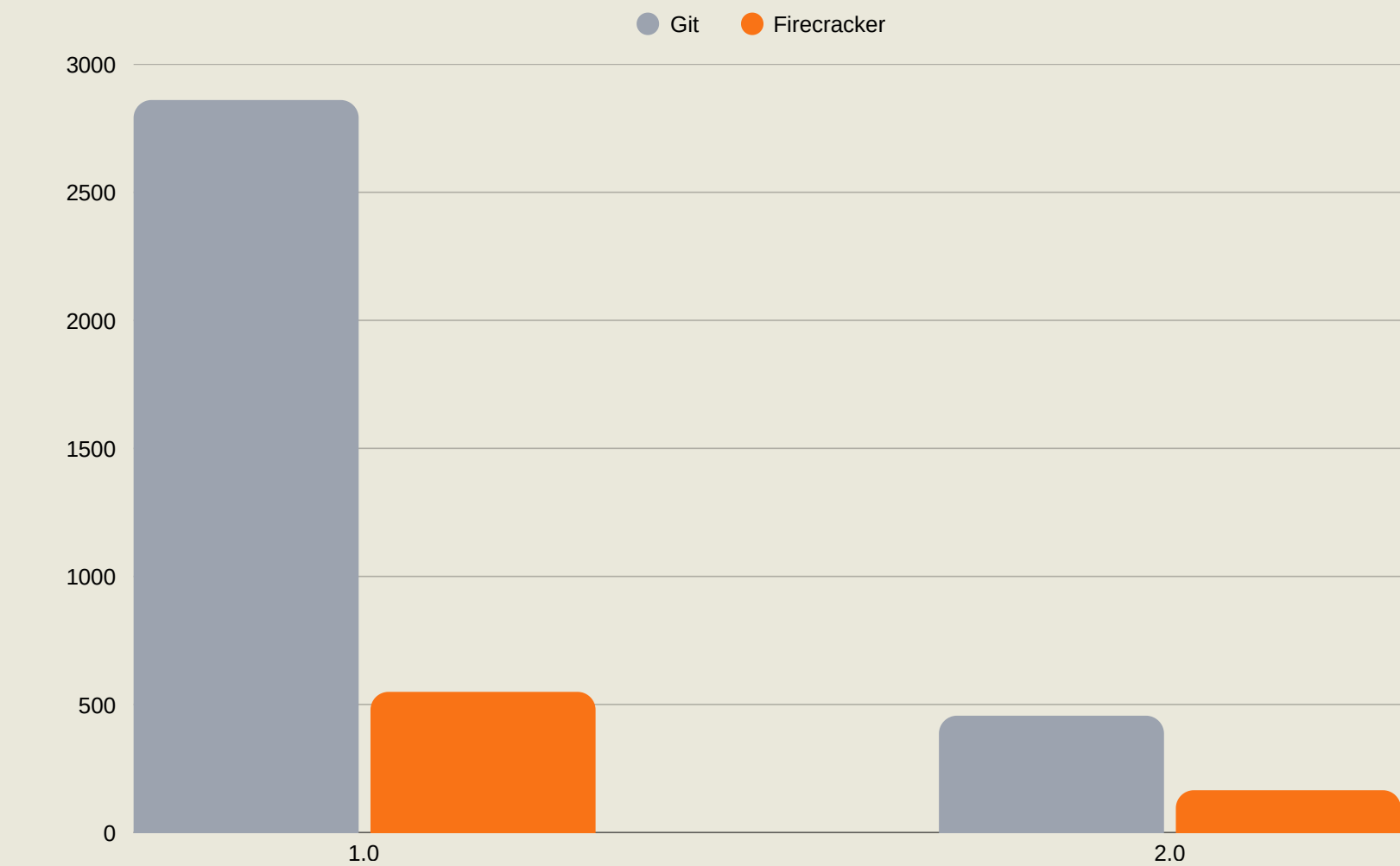


Restore Latency
(V2 Live)

Git: 6.57s **FC: 0.18s**

Penalty Time
(LLM Repair)

Git: 6.57s **FC: 0.00s**



Token
Consumption

Git: 2860 **FC: 549**

Context Pollution

Git: 456.1 **FC: 165.6**

Exp 3: Otimização Incremental

Snapshot Full (Armazenamento)



Snapshot Diff (SnapVM)



Redução de I/O

-36.5% no footprint de armazenamento, permitindo checkpoints de alta frequência.

Velocidade de Captura

-19% na latência de pausa da MicroVM durante o salvamento de estado.

Exp 4: Recuperação Autônoma

-40% Consumo de Tokens

1.72s Latência Média

85% Snapshot Restore

Conclusão do JSON v4: Sem coaching, a IA identifica o ``restore_last_snapshot`` como o caminho de menor custo cognitivo, evitando a investigação manual de logs corrompidos.

Exp 4.1: Falhas Multi-Camada

ID	Cenário de Falha	Complexidade
F1	Corrupção Multi-Tabela + Dados Rogue	Investigação Ocupada
F2	Kill Process + Drop Table + Config Loss	Cascata de Erros
F3	Schema Alteration + Invalid Data Injection	Silenciosa / Semântica
F4	Drop Table + Revoke Permissions	Bloqueio de Reparo
F5	Truncate + Service Kill + Socket Removal	Falha Crítica Total

O Desafio da Opacidade

O endpoint de /health reporta apenas o primeiro erro. A IA deve investigar as causas secundárias ocultas.

- **Standard:** Tenta reparar um problema por vez (Alto Custo).
- **SnapVM:** Restaura o estado bit-perfect global (Custo Fixo).

Exp 4.1: Custo de **Investigação**

F1: Multi-table Corruption (Tokens)

Std: 1751

Snap: 1688

F2: Process & Config (Tokens)

Std: 1758

Snap: 1036

F4: Permissions (Tokens)

Std: 1757

Snap: 1514

Insight Crítico: Psicologia do Agente. Em cenários "opacos", a IA tende a tentar o reparo manual antes de restaurar o snapshot se o erro parecer simples. Isso revela que o SnapVM precisa ser integrado à **lógica de prioridade** da IA para maximizar economia.

Exp 4.2: Eficiência Máxima

Otimização Determinística

Ao forçar a política de "**Snapshot First**", eliminamos a variável psicológica da IA.

-67%

Redução no consumo de tokens em todos os cenários complexos (F1-F5).

0.92s

Latência constante de recuperação ($O(1)$) independente da falha.

Cenário	Std Tokens	Snap Tokens	Ganho
F1	1758	449	3.9x
F2	1758	449	3.9x
F4	1757	449	3.9x
F3/F5	~740	431	1.7x

Exp 5: **Proatividade** do Agente

Cenário Real

Migration Workflow de 6 passos.
Injeção de falhas nos passos 2 e 4.

100%

Taxa de conclusão vs 67% no
Baseline Standard.

Auto-Gestão

A IA usou save_checkpoint
autonomamente antes de
migrações arriscadas.

Labels de Checkpoint Criados pela IA:

- "Initial state before migration"
- "Products table created and seeded"
- "Categories table created..."

O Snapshot deixa de ser uma ferramenta de recuperação
e torna-se um **primitivo de desenvolvimento**.

Conclusão Experimental

Validação da Tese

A orquestração baseada em snapshots fornece um tempo de recuperação **O(1)** para o ambiente, removendo a carga cognitiva da IA em falhas de infraestrutura.

Impacto Científico

A redução de **5.2x no consumo de tokens** e de **49% na poluição de contexto** comprova que a resiliência agentic é, primariamente, um problema de infraestrutura física.

Os agentes são espertos o suficiente para usar um "Save Button" se você fornecer isso a eles!

Equipe SnapVM: Arthur Carvalho, Débora Luiza, Gustavo
Henrique e Mariana Almeida

PUC Minas - Ciência da Computação | 2026